

Detector de Fraude Transaccional — XGBoost

Contenido

1. Planteamiento del Problema	1
2. Consideraciones del Modelo	1
3. Narrativa del Modelo	2
4. Selección y Justificación del Punto de Corte (Umbral de Decisión)	3
5. Consideraciones para Producción	4
6. Consideraciones Adicionales	5

1. Planteamiento del Problema

En la industria financiera y de procesamiento de pagos, la prevención y mitigación de fraudes es un pilar crítico. El problema principal reside en identificar, en tiempo real o en lotes de corta latencia, si una solicitud transaccional debe ser aprobada o declinada debido a un riesgo inminente de fraude.

Los principales retos identificados en el dataset `datos_fraud.csv` son los siguientes:

Desbalance Severo de Clases: De las 284,807 transacciones registradas, únicamente 492 corresponden a eventos de fraude (aproximadamente el 0.17% del total de las observaciones). Este desbalance extremo renderiza inútiles las métricas tradicionales de evaluación como la exactitud, ya que un modelo trivial que califique todas las transacciones como legítimas alcanzaría un 99.83% de exactitud, fallando completamente en capturar la clase de interés.

Estructura de datos desconocida: Se cuenta con variables numéricas anónimas (resultado de transformaciones de reducción de dimensionalidad o anonimización), junto con el identificador único (`transaction_id`), la marca de tiempo en segundos (`timestamp`) y el monto (`amount`). La mayoría de las variables anónimas presentan distribuciones leptocúrticas con datos atípicos severos en las colas.

2. Consideraciones del Modelo

Para abordar el problema planteado y maximizar el rendimiento discriminativo sobre la clase minoritaria, se definieron las siguientes directrices metodológicas y técnicas:

Selección de la Métrica Objetivo (PR-AUC vs. ROC-AUC)

Dada la naturaleza desbalanceada del fenómeno, la curva Precision-Recall (PR-AUC / Average Precision) se fijó como la métrica primaria de optimización. A diferencia de ROC-AUC, que utiliza la Tasa de Falsos Positivos y se diluye debido al elevado número de Verdaderos Negativos, PR-AUC evalúa directamente la interacción entre Precision (exactitud de la alerta de fraude) y Recall (cobertura total del fraude), reflejando con mayor fidelidad el impacto real en producción.

Preprocesamiento

Se validó la presencia de transacciones duplicadas: Son transacciones completamente idénticas que no añadían información y que sesgaban la muestra por lo que fueron eliminadas.

Tratamiento del Monto (amount): Se aplicó una transformación logarítmica para reducir el sesgo a la derecha y suavizar la influencia de transacciones con montos atípicamente altos. Asimismo, se generaron flags binarias para identificar micro-transacciones (es_uno_o_menos, es_cero) que suelen usarse para prueba de tarjetas.

Cortes de Montos por Deciles: Se implementó una discretización en 10 deciles sobre amount_log.

Transformaciones Temporales Cíclicas: A partir de la marca de tiempo en segundos (timestamp), se extrajo la hora del día, el día de la transacción y una variable indicadora de horario nocturno/crítico (is_night_transaction para transacciones entre las 00:00 y las 05:59 hrs). Para evitar saltos discontinuos entre las 23:59 y las 00:00, la hora fue codificada en componentes trigonométricas seno y coseno.

División Estratificada de Datos: Se realizó una partición de los datos en 80% Entrenamiento y 20%.

Selección de Algoritmos Evaluados

Se evaluaron tres arquitecturas:

- Random Forest: Configurado con pesos de clase ajustados (class_weight='balanced').
- LightGBM: Evaluado con parámetros de ponderación de instancias (is_unbalance=True y scale_pos_weight).
- XGBoost: Ajustado mediante la relación de clases negativas a positivas y función de pérdida optimizada.

3. Narrativa del Modelo

Exploración y Benchmark Inicial

En la fase inicial sin afinación fina, los tres modelos fueron entrenados sobre el conjunto de datos procesado. XGBoost demostró desde las primeras iteraciones una capacidad superior para capturar la estructura de las variables latentes y manejar el desbalance mediante scale_pos_weight, obteniendo métricas sobresalientes de PR-AUC en comparación con Random Forest y LightGBM.

Optimización de Hiperparámetros (Optuna)

Se ejecutó una búsqueda bayesiana de hiperparámetros utilizando Optuna, fijando como función objetivo la maximización del PR-AUC a través de una validación cruzada estratificada de 5 particiones. Los hiperparámetros resultantes de mayor impacto para XGBoost incluyeron:

- **n_estimators:** 1500 (con parada temprana / early stopping).
- **learning_rate:** 0.0389
- **max_depth:** 6 (para controlar la profundidad de interacción de variables sin generar sobreajuste).
- **subsample:** 0.8001 y **colsample_bytree:** 0.7108 (muestreo estocástico de filas y columnas por árbol).
- **scale_pos_weight:** 223.94 (escalado ponderado del gradiente para la clase positiva).
- **reg_alpha (L1):** 2.0069 y **reg_lambda (L2):** 3.4773 (regularización robusta).
- **gamma:** **3.4233** (reducción mínima de pérdida requerida para realizar una nueva partición).

Evaluación Comparativa de Ensamble vs. Modelo Individual

Se evaluó la viabilidad de implementar un ensamble ponderado (90% XGBoost + 10% Random Forest). Aunque el ensamble demostró una curva suave, el modelo individual XGBoost Optimizador mantuvo el mejor desempeño global, menor complejidad computacional en inferencia y un excelente control de sobreajuste.

Diagnóstico de Ajuste y Generalización (Train vs. Test)

Para descartar problemas de memorización u overfitting, se compararon las métricas de entrenamiento frente al conjunto de validación/prueba:

Figura 1: Comparativa entre entrenamiento y testeo

Métrica Evaluada	Entrenamiento (X _{tr})	Prueba (X _{test})	Gap Generalización (Train - Test)
PR-AUC (Métrica Principal)	0.8931	0.8524	0.0407
ROC-AUC (Métrica Secundaria)	0.9912	0.9815	0.0097

Veredicto: El gap en PR-AUC es de 0.0407 (4.07%), el cual se encuentra por debajo del umbral de tolerancia del 5%. Esto confirma una excelente capacidad de generalización frente a datos no observados previamente.

4. Selección y Justificación del Punto de Corte (Umbral de Decisión)

Por defecto, los clasificadores binarios asignan el umbral de decisión en **0.5**. Sin embargo, en un entorno de desbalance severo con ponderación de costo, utilizar 0.5 destruye la precisión o la cobertura según el ajuste.

Determinación Imparcial Out-of-Fold (OOF)

Para evitar sesgos y fuga de información del conjunto de testing, el umbral óptimo se calculó estrictamente fuera de muestra sobre el conjunto de entrenamiento mediante validación cruzada estratificada y la optimización del F_1 -Score/ F_2 -Score sobre la curva Precision-Recall.

El punto de corte óptimo obtenido fue: **OPTIMAL_THRESHOLD = 0.8878**

Resumen de Métricas Operativas en Producción

Al aplicar de manera transparente la clase contenedora ProductionFraudClassifier con el umbral de 0.8878 sobre el conjunto de prueba desacoplado, se obtuvieron los siguientes resultados:

Métrica Operativa	Valor Obtenido	Interpretación de Negocio
Umbral Aplicado	0.8878	Punto de corte probabilístico final para marcar la alerta de fraude.
Precisión (Precision)	86.32%	De cada 100 alertas emitidas por el modelo, ~86 son fraudes reales y solo ~14 son falsas alarmas.
Cobertura (Recall)	83.67%	El modelo captura e intercepta exitosamente el 83.67% de todas las pérdidas por fraude.
F1-Score	0.8498	Media armónica sólida entre precisión y cobertura.

<i>F2-Score</i>	0.8419	Métrica ponderada otorgando doble importancia a la detección de fraudes (Recall).
<i>Tasa de Falsos Positivos (FPR)</i>	0.0228%	Únicamente 13 clientes legítimos de 56,864 sufrieron fricción o revisión manual.

Matriz de Confusión y Desglose de Impacto Financiero en Test

Sobre un total de 56,962 transacciones de prueba evaluadas:

<i>Clase Real</i>	<i>Predicción del Modelo</i>	<i>Tipo de Resultado</i>	<i>Conteo de Casos</i>	<i>Porcentaje Relativo</i>
<i>Legítimo (0)</i>	Aprobado (0)	Verdadero Negativo (TN)	56,851	99.977%
<i>Legítimo (0)</i>	Bloqueado / Alerta (1)	Falso Positivo (FP)	13	0.023%
<i>Fraude (1)</i>	No Detectado (0)	Falso Negativo (FN)	16	16.327%
<i>Fraude (1)</i>	Capturado (1)	Verdadero Positivo (TP)	82	83.673%

5. Consideraciones para Producción

Poner en producción un modelo de detección de fraude requiere un marco arquitectónico resiliente que garantice inferencias de baja latencia y alta confiabilidad operativa.

Empaquetamiento y Artefacto de Producción

El artefacto desplegable final fue encapsulado en la clase customizada `ProductionFraudClassifier` y serializado en formato binario comprimido mediante `joblib` en la ruta `Salidas/Modelos_finales/modelo_fraude_xgboost_final.joblib`.

Arquitectura de Despliegue

Microservicio REST / gRPC API: Encapsulamiento del modelo dentro de un contenedor Docker utilizando FastAPI o Triton Inference Server.

Validación de Esquema (Data Validation): Inclusión de un contrato de datos estricto con Pydantic en la capa de entrada para validar tipos de datos, rangos y valores nulos antes de pasar las variables al pipeline de inferencia.

Detección de Data Drift y Concept Drift:

Monitoreo de la Distribución del Score: Registrar en bases de datos de series temporales la distribución del score de salida predicho. Si la mediana del score se desplaza hacia la derecha, el modelo podría estar reaccionando a un sesgo o cambio en los patrones transaccionales.

Estrategia de Shadow Deploy / A-B Testing: Antes de reemplazar por completo un modelo existente, desplegar la nueva versión en modo sombra recibiendo el 100% del tráfico para comparar sus alertas frente a la herramienta actual sin impactar las transacciones reales.

Reentrenamiento Automatizado: Establecer un pipeline de reentrenamiento periódico (mensual o trimestral) o desencadenado por eventos cuando las métricas de monitoreo de drift sobrepasen umbrales de tolerancia predefinidos.

6. Consideraciones Adicionales

Persistencia de Metadatos

Como parte de los estándares de gobernanza de datos y trazabilidad del modelo, se estructuró un archivo normalizado `model_metadata.json` guardado en la ruta de salidas. Este JSON contiene:

- Identificación de versión del modelo (1.0.0) y framework utilizado.
- Esquema de variables de entrada y su importancia.
- Registro de hiperparámetros óptimos utilizados durante el entrenamiento.
- Resultados consolidados de métricas discriminativas y matriz de confusión sobre el set de prueba.

Gobernanza y Explicabilidad

Debido a regulaciones financieras (tales como el derecho a la explicación de decisiones automatizadas de crédito/riesgo), se recomienda incorporar valores SHAP en el microservicio de inferencia para generar los principales 3 factores que contribuyeron al score de cada transacción marcada como fraude en los paneles de auditoría.